

From
Code to
Deployment

GitLab 101

A PRACTICAL GUIDE FOR DEVOPS ENGINEERS

SOURCE CODE | CI/CD | SECURITY | DEPLOYMENTS | DEVSECOPS



Code



CI/CD



Security



Container
Registry



GitLab®



Collaboration



Deployments



Automation



Monitor
& Improve

*Faster Software
Safer Systems
Stronger Teams*

20 PAGES

Plan
Build
Test
Secure
Deploy
Operate
Repeat

Better
Engineers
Build
Better
Systems



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

GitLab Architecture and Mental Model

1. What GitLab Actually Provides

GitLab is a complete DevSecOps platform that helps you plan, develop, secure, build, deploy and operate software in one place.



Source Control



CI/CD



Container Registry



Security



Operations & Observability

2. Projects, Groups and Namespaces

- **Project:** A single repository and its configuration (CI/CD, issues, etc).
- **Group:** A container for multiple projects.
- **Subgroup:** Organize projects at scale.
- **Namespace:** Unique path for groups and projects (e.g. veriqta/platform/api).
- **Visibility:** Private, Internal, Public.
- Used for access control, organization and ownership at scale.

3. GitLab as a DevSecOps Platform

GitLab unifies the software development lifecycle with built-in security and operations. This reduces tool sprawl and improves collaboration, visibility and delivery speed.

4. Key Components

Repository	Git-based source code management
CI/CD	Build, test and deploy pipelines
Registry	Store container images and packages
Security	SAST, DAST, dependency and secret scanning
Operations	Monitor, manage and operate applications

5. Control Plane vs Workload Execution

Control Plane (GitLab Instance)

Web UI
API
CI/CD Coordinator
Database
Background Workers

Manages configuration, stores data, coordinates pipelines and schedules jobs.

Sends jobs to runners

Reports job status, logs and artifacts

Workload Execution (Runners)

Shell Runner

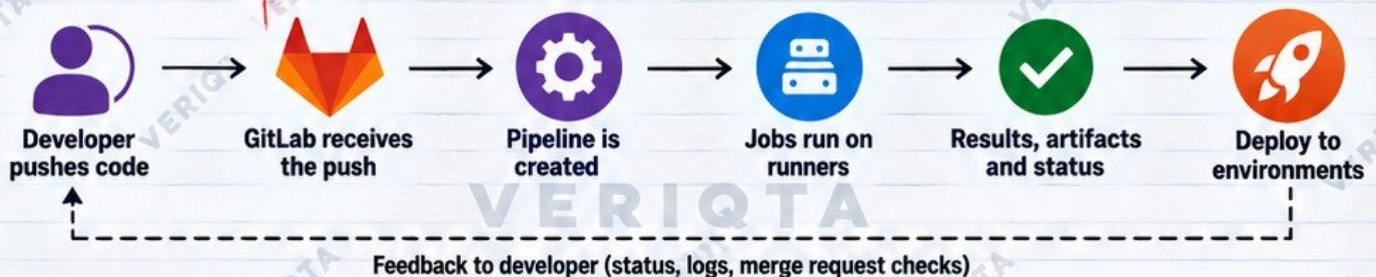
Docker Runner

Kubernetes Runner

Cloud Runners

Runners execute CI/CD jobs in isolated environments and return results to GitLab.

6. High-Level Request and Pipeline Flow



Key Takeaway

GitLab is more than a Git repository. It is an integrated platform that connects source code, CI/CD, security and operations, giving teams a single place to build and run software securely at scale.



Real World Benefit

- Faster delivery
- Better security
- Improved collaboration
- Less tool sprawl
- End-to-end visibility



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

Projects, Groups, and Namespace Design

1. Project Structure

- A project is a single repository with its own settings, CI/CD, issues, etc.
- Projects belong to a namespace (user or group).
- Each project can have different dependency, pipeline and access rules.

2. Groups and Subgroups

- Groups organize multiple projects.
- Subgroups allow hierarchical structure (e.g. department → team → projects).
- Groups make it easier to manage permissions, CI/CD settings and shared resources.

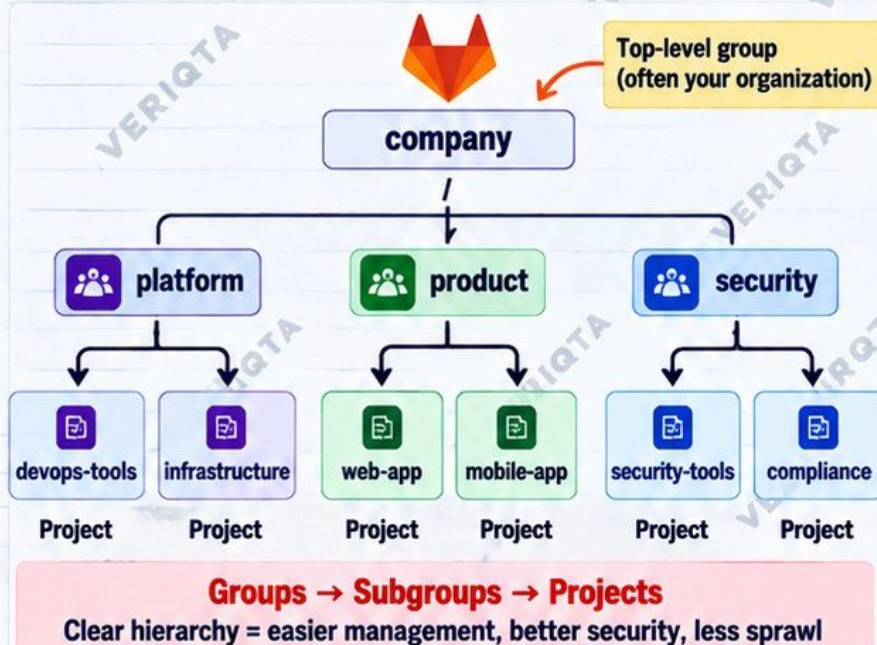
3. Namespace Hierarchy

- A namespace can be a user or a group.
- Groups and subgroups create a hierarchy.
- Example:

company / platform / backend / api-service

- Use namespaces to reflect your organization structure.

4. Example Namespace Structure



5. Visibility Levels

Level	Who can access
Private	Only explicitly invited users
Internal	Any logged-in user in your GitLab instance
Public	Anyone (no login required)

6. Ownership Boundaries

- Use groups to map to real teams or departments.
- Assign group owners and maintainers carefully.
- Separate ownership for applications, platform, and infrastructure.
- Use group-level settings and permissions to enforce boundaries.

7. Designing for Large Organizations

- ✓ Use a clear and consistent group structure.
- ✓ Align namespaces with your organization (e.g. business unit → team → project).
- ✓ Use templates for new projects.
- ✓ Centralize CI/CD, security policies and runner configuration at group level.
- ✓ Apply naming conventions (e.g. <team>-<service>-<environment>).
- ✓ Document ownership and maintainers.
- ✓ Use parent groups for shared resources and standards.

⚠ Avoiding Permission Sprawl

- ✗ Do not give wildcard or unnecessary maintainer access.
- ✗ Use least privilege principle.
- ✗ Regularly review group and project members.
- ✗ Remove inactive users.
- ✗ Use groups instead of direct project access for easier management.
- ✗ Automate access reviews where possible.

Key Takeaway: A well-designed GitLab structure improves security, collaboration and scalability.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

GitLab Repository Workflow



1. Clone, Fetch, Pull, Push

```
# Clone a repository
git clone <gitlab-url>

# Fetch latest changes (no merge)
git fetch origin

# Pull latest changes (fetch + merge)
git pull origin <branch>

# Push your changes
git push origin <branch>
```

2. Remote Repositories

- In GitLab, the remote is usually named **origin**.
- It points to your GitLab project URL (HTTPS or SSH).
- You can have multiple remotes (e.g. upstream, backup).
- Check remotes:

```
git remote -v
```

3. Default Branches

- The default branch is the **main** branch for development (e.g. **main**).
- New merge requests target this branch.
- You can change the default branch in:

Settings → Repository
→ Default branch

4. Protected Branches

- Protect important branches (e.g. **main**, release) from unauthorized changes.
- Control who can push, merge and create merge requests.
- Common protections:
 - Only allow merge requests
 - Require approvals
 - Restrict direct pushes
 - Enforce status checks (CI pipelines)

5. Tags

- Tags mark specific releases (e.g. **v1.0.0**).
- Used for versioning and production releases.
- Create a tag:

```
git tag v1.0.0
git push origin v1.0.0
```

- List tags:

```
git tag -l
```

- Delete a tag:

```
git push origin --delete v1.0.0
```

6. Repository Settings

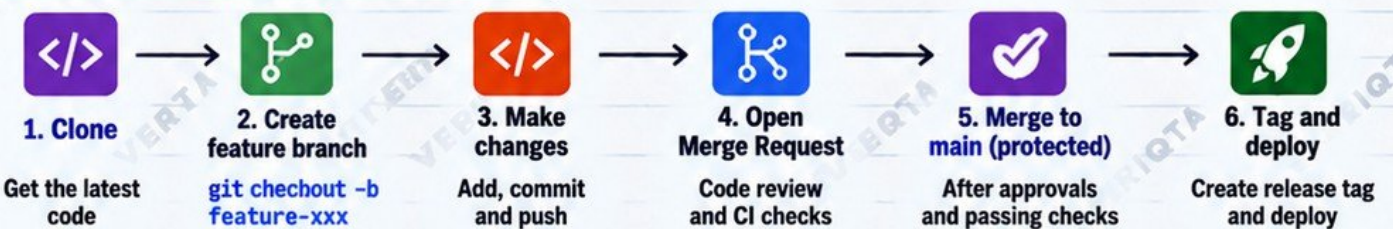
- Access repository settings in:

Settings → Repository

- Key settings:

- Default branch
- Protected branches
- Tags
- Merge request options
- Repository visibility
- Code owners
- Large file storage (LFS)
- Webhooks
- Push rules

7. Production-Safe Git Workflow



Best Practices

- Use feature branches.
- Protect **main** and release branches.
- Require merge requests and approvals.
- Use tags for releases.
- Keep commit messages clear and meaningful.
- Use CI pipelines to validate changes.
- Avoid direct pushes to protected branches.



Key Takeaway

A well-structured Git workflow in GitLab improves collaboration, code quality and release safety. Use protected branches, merge requests and tags to keep your repository secure and production-ready.



Common Mistakes

- Pushing directly to main.
- Not using merge requests.
- No branch protection.
- Poor commit messages.
- Skipping code reviews.
- No tags for releases.
- Giving excessive permissions.
- Ignoring repository settings.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

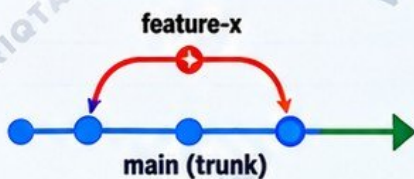
Branching Strategies in GitLab



CHOOSE A STRATEGY THAT MATCHES YOUR TEAM, RELEASE PROCESS AND DEPLOYMENT ARCHITECTURE

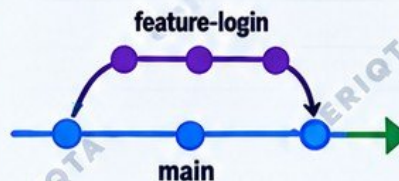
1. Trunk-Based Development

- Developers work on short-lived branches and merge directly to main (trunk).
- Frequent commits and small changes.
- Relies on strong CI/CD and automated tests.
- Best for teams that deploy frequently (e.g. cloud-native, microservices).



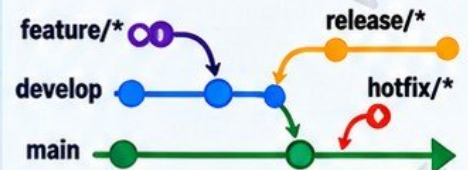
2. Feature Branches

- Create a new branch for each feature.
- Isolates work and reduces risk on main.
- Merge using merge requests (after review and CI checks).
- Keep branches short-lived.



3. GitFlow

- Uses multiple long-lived branches for structure.
- main**: production-ready code.
- develop**: integration branch.
- feature/***: new features.
- release/***: prepare for release.
- hotfix/***: urgent fixes.



4. Release Branches

- Used to prepare a stable release (e.g. v1.0.0).
- Only bug fixes and minor changes allowed.
- After testing, merged into **main** and **develop** (if using GitFlow).



5. Hotfix Workflows

- Used for urgent production fixes.
- Create a hotfix branch from **main** (or production branch).
- Make the fix, test, and merge back to **main** (and **develop** if using GitFlow).
- Tag the release.



6. Long-Lived Branch Problems

- ✗ High merge conflicts.
- ✗ Code becomes stale.
- ✗ Harder to integrate changes.
- ✗ More time spent resolving conflicts.
- ✗ Increases risk and slows delivery.
- ✗ Developers work on outdated code.
- ✗ Creates multiple versions of the truth.

Better Approach:
Keep branches short-lived and merge frequently.

7. Choosing a Branching Model

Consider your deployment architecture:

Scenario / Architecture	Recommended Strategy
Continuous deployment (e.g. microservices)	Trunk-based development
Regular releases (e.g. web application)	Feature branches or GitFlow
Strict release cycles (e.g. regulated industries)	GitFlow with release branches
Small team / simple setup	Feature branches or trunk-based
Multiple environments (dev, staging, prod)	Feature branches with environment-based deployments



Key Takeaways

- ✓ Choose a strategy that fits your team and deployment model.
- ✓ Use merge requests for code review and CI checks.
- ✓ Keep branches short-lived.
- ✓ Automate testing and deployments.
- ✓ Use protected branches for **main** and release branches.
- ✓ Document your branching strategy for the whole team.



Best Practices

- ✓ Use meaningful branch names (e.g. feature/payment-api).
- ✓ Protect **main** and release branches in GitLab.
- ✓ Require merge request approvals.
- ✓ Automate CI/CD pipelines.
- ✓ Delete merged branches.
- ✓ Use tags for releases.
- ✓ Keep the workflow simple and consistent.



Remember

Your branching strategy should improve collaboration and release safety, not create complexity.

“ A good branching strategy enables fast delivery, high code quality, and a stable production environment. ”



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

Merge Requests and Production Code Review



BETTER REVIEWS. SAFER CHANGES. STRONGER SYSTEMS.

1. Merge Request Lifecycle



2. Reviewers and Approvals

- Assign reviewers (individuals or groups).
- Use required approvals for critical code.
- Use **CODEOWNERS** for automatic reviewer assignment.
- Set different approval rules for different branches (e.g. **main**).
- Use group approvals for large teams.
- Ensure separation of duties for production changes.

3. Draft Merge Requests

- Use draft MRs for work in progress.
- Clearly mark as Draft.
- Useful for early feedback and CI runs.
- Not ready for formal review or approval.
- Convert to a regular MR when ready.

 **Draft:** Add user authentication

4. Discussions

- Use the discussion thread to ask questions and give context.
- Comment on specific lines of code.
- Resolve conversations when done.
- Keep discussions professional and actionable.
- Use @mentions to involve the right people.

5. Merge Conflicts

- Happen when changes conflict between branches.
- Resolve conflicts locally:

```
git fetch origin
git checkout <branch>
git merge <target-branch>
```

- Fix conflicts, commit and push.
- Use GitLab's web UI for simple conflicts.

6. Squash and Rebase

- **Squash:** Combines all commits into one. Keeps history clean.
- **Rebase:** replays commits on top of the latest target branch.
- Use **squash** for feature branches before merging.
- Use rebase to keep a linear and clean history.

7. Merge Strategies

Strategy	Description
Merge commit	Creates a merge commit. Preserves full history.
Squash and merge	Combines commits into one. Cleaner history.
Rebase and merge	Replays commits on top of target branch. Linear history.

8. CODEOWNERS

- Define owners for specific files or directories.
- Automatically requests reviews from the right team.
- Helps enforce ownership and accountability.
- Example **.gitlab/CODEOWNERS**:

```
# Global owners
* @platform-team
# Application code
/apps/ @app-team
/infrastructure/ @infra-team
```

9. Approval Gates and Separation of Duties

- Use required approvals for **main** and release branches.
- Enforce different approval levels for sensitive changes.
- Separate code authors, reviewers, and deployers.
- Use protected branches and merge rules.
- Integrate CI/CD checks (tests, security scans) as gates.
- Consider change advisory process for production.



Key Takeaway

Merge requests are not just about merging code. They are a control point for quality, security, ownership and safe delivery to production.



Common Mistakes

- Merging without proper review.
- Not using **CODEOWNERS**.
- Too many long discussions.
- Ignoring failing pipeline checks.
- Overusing "squash" without understanding impact.
- No clear approval rules.
- Letting the same person code, review and deploy.



Good code gets features shipped. Great code reviews keep systems running.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

GitLab CI/CD Architecture

FROM CODE TO DEPLOYMENT. AUTOMATED. SECURE. SCALABLE.



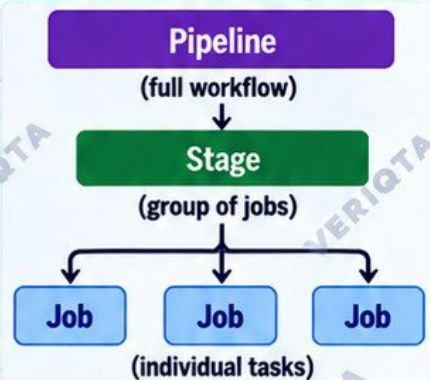
1. What is GitLab CI/CD?

- GitLab CI/CD is a built-in automation engine to build, test and deploy your applications.
- Defined using a `.gitlab-ci.yml` file in your repository.
- Runs jobs in stages using runners.
- Helps you ship code faster, safer and more reliably.

2. Key Components

Component	Description
Pipeline	A collection of jobs that run in stages.
Job	A single task (e.g. build, test).
Stage	A group of jobs that run in sequence.
Runner	A machine that executes jobs.
Executor	The environment used by the runner (e.g. shell, docker).
<code>.gitlab-ci.yml</code>	Configuration file that defines your pipeline.

3. Pipeline, Job and Stage



4. GitLab Runner and Executor



GitLab Runner

- Runner picks up jobs from GitLab and executes them.
- Runners can be **shared**, **group** or **project** specific.
- Executors** define the environment for running jobs.

Common Executors



Shell

(runs on host)



Docker

(container)



Kubernetes

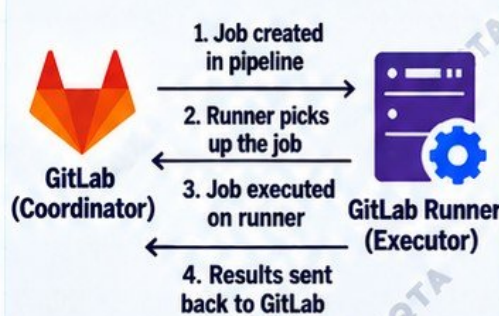
(pod)



Custom

(e.g. cloud VMs)

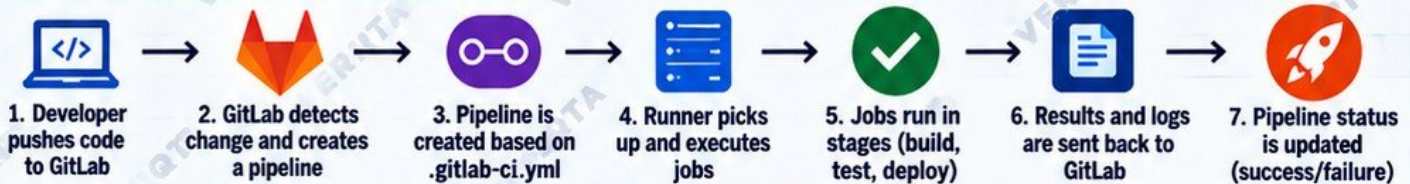
5. Coordinator-to-Runner Flow



The coordinator:

- Creates pipelines
- Stores configuration
- Schedules jobs
- Sends jobs to runners
- Collects results
- Updates pipeline status

6. What Happens After git push?



7. Example .gitlab-ci.yml (Simple)

```

stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - echo "Building the application..."

test:
  stage: test
  script:
    - echo "Running tests..."

deploy:
  stage: deploy
  script:
    - echo "Deploying to production..."
  
```

8. Best Practices

- Keep your `.gitlab-ci.yml` in the root of the repository.
- Use clear stages and meaningful job names.
- Use runner tags and protected runners.
- Store secrets in CI/CD variables (not in the file).
- Use environments for deployments.
- Add proper logging and artifacts.
- Keep pipelines fast and reliable.

Key Takeaway

GitLab CI/CD automates your entire delivery process from code to production using pipelines, jobs, runners and a simple configuration file. Understanding this architecture helps you build secure, scalable and efficient delivery workflows.

Common Mistakes

- Missing or incorrect `.gitlab-ci.yml`.
- Using unprotected runners for sensitive jobs.
- Storing secrets in the repo.
- Long-running jobs with no timeout.
- Not using stages properly.
- Ignoring pipeline failures.
- No environment protection for production.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

Building Your First Production-Style Pipeline



AUTOMATE. TEST. PACKAGE. DEPLOY. REPEAT.

1. What is .gitlab-ci.yml?

- .gitlab-ci.yml defines your CI/CD pipeline.
- It lives in the root of your repository.
- It tells GitLab what to do, in which order, and using which runners.
- Use YAML syntax.
- Commit and push it to trigger pipelines automatically.

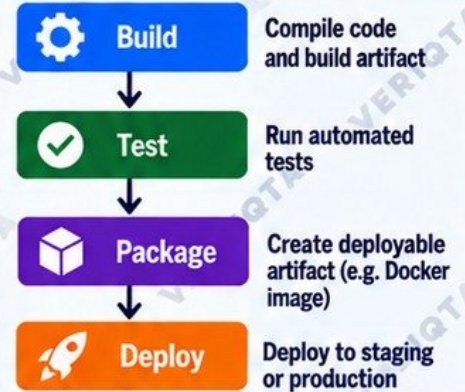


File name must be:
.gitlab-ci.yml

2. Key Keywords

Keyword	Purpose
stages	Defines the pipeline stages.
script	Commands to run in a job.
before_script	Runs before each job's script.
after_script	Runs after each job's script.
jobs	Individual tasks in the pipeline.
needs	Defines job dependencies.
artifacts	Stores files from a job.
rules	Controls when jobs run.
variables	Defines environment variables.

3. Pipeline Stages Example



4. Example .gitlab-ci.yml

```

1 stages:
2   - build
3   - test
4   - package
5   - deploy
6
7 before_script:
8   - echo "Setting up environment..."
9
10 build:
11   stage: build
12   script:
13     - echo "Building the application..."
14     - make build
15   artifacts:
16     paths:
17       - build/
18
19 test:
20   stage: test
21   script:
22     - echo "Running tests..."
23     - make test
24   needs: ["build"]
25
26 package:
27   stage: package
28   script:
29     - echo "Creating Docker image..."
30     - docker build -t myapp:SCI_COMMIT_SHORT_SHA .
31     - docker push SCI_REGISTRY_IMAGE:SCI_COMMIT_SHORT_SHA
32
33 deploy:
34   stage: deploy
35   script:
36     - echo "Deploying to production..."
37     - ./deploy.sh
38   environment:
39     name: production
40     needs: ["package"]
  
```

5. Job Dependencies (needs)

- Use **needs** to run jobs in parallel or specify dependencies.
- A job can start as soon as its dependencies are complete.
- This makes pipelines faster.

Example:

```

test:
  needs: ["build"]
  
```

6. before_script and after_script

before_script

- Runs before each job's script.
- Useful for installing dependencies, setting up environment, etc.

Example:

```

before_script:
  - echo "Installing tools..."
  - apt-get update
  
```

after_script

- Runs after each job's script.
- Useful for cleanup, notifications, etc.

Example:

```

after_script:
  - echo "Cleaning up..."
  
```

7. Pipeline Status

-  **Success** All jobs passed.
-  **Failed** At least one job failed.
-  **Running** Pipeline is in progress.
-  **Pending** Waiting for a runner.
-  **Canceled** Pipeline was canceled.
-  **Skipped** Job was skipped (by rules or conditions).

8. Build → Test → Package → Deploy Flow



Best Practices

- Keep .gitlab-ci.yml in the root of the repository.
- Use clear and consistent stage names.
- Break complex pipelines into smaller jobs.
- Use artifacts and caching to improve speed.
- Use **needs** for faster pipelines.
- Store sensitive values in CI/CD variables.
- Test your pipeline changes in a feature branch.



Key Takeaway

A well-structured pipeline automates your build, test, package and deploy process, makes your releases faster and safer, and gives you confidence in every change you ship to production.



Common Mistakes

-  Wrong YAML syntax (indentation).
-  Missing or incorrect stages.
-  Jobs running in the wrong order.
-  Not using artifacts or caching.
-  Hardcoding secrets in the file.
-  Forgetting environment setup.
-  Not testing the pipeline before merging to **main**.

LEARN
PRACTICE
BUILD
GROW



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

GitLab Runners Explained



RUN JOBS. SCALE WORKFLOWS. KEEP IT SECURE.

1. What is a GitLab Runner?

- A GitLab Runner is a lightweight agent that picks up CI/CD jobs from GitLab and executes them.
- Runners run jobs in an isolated environment using an executor (e.g. shell, Docker, Kubernetes).
- You can use **shared runners** (GitLab-hosted) or set up your own (project or group runners).

Runners **do not store your code**. They only **execute jobs**.

2. Shared vs Project/Group Runners

Type	Description	Pros	Cons
Shared Runners	Provided by GitLab (SaaS or self-hosted) and shared across all users.	No setup required Free (on GitLab.com) Good for learning and small projects	Limited control Possible queue wait times Not suitable for sensitive jobs
Project Runners	Dedicated to a single project.	More control Custom configuration Can use private infrastructure	You manage them Requires setup and maintenance
Group Runners	Shared across all projects in a group.	Centralized management Useful for large organizations	Requires setup and governance

3. Runner Registration

- Register a runner using a token from GitLab (project, group or instance).
- Choose an executor (e.g. shell, docker, kubernetes).
- Assign tags (optional).
- Start the runner service.

```
gitlab-runner register
--url https://gitlab.com
--token <REGISTRATION_TOKEN>
--executor docker
--description "my-runner"
--tag-list "docker,linux"
--run-untagged=false
```

4. Executors

An executor defines the environment in which the job runs.

Executor	Description	Use Case
Shell	Runs jobs directly on the host machine.	Simple setup, trusted environments
Docker	Runs jobs inside Docker containers.	Isolated, portable, most common
Kubernetes	Runs jobs in Kubernetes pods.	Scalable, cloud-native environments

Choose the right executor based on your infrastructure, security and scalability needs.

5. Shell Executor



- Runs jobs directly on the host OS.
- Fast and simple.
- No container isolation.
- Best for trusted environments.

Example (config):
executor = "shell"

6. Docker Executor



- Runs jobs inside Docker containers.
- Clean and isolated environment.
- Supports custom images.
- Commonly used for most CI/CD pipelines.

Example (config):
executor = "docker"
image = "alpine:latest"

7. Kubernetes Executor



- Runs jobs in Kubernetes pods.
- Highly scalable.
- Ideal for large organizations.
- Supports auto-scaling and dynamic environments.

Example (config):
executor = "kubernetes"

8. Runner Tags

- Tags help match jobs with specific runners.
- Use tags to ensure jobs run on the right infrastructure (e.g. docker, linux, gpu).

```
# In .gitlab-ci.yml
job:
  tags:
    - docker
    - linux
```

9. Concurrency

- Defines how many jobs a runner can run at the same time.
- Helps control resource usage and performance.

```
# In config.toml
concurrent = 4
```

10. Isolation

- Use containers or VMs for isolation.
- Avoid running untrusted code on shared runners.
- Use dedicated runners for sensitive projects.
- Combine with tags and protected branches.

11. Why Runner Security Matters

- Runners execute your code and can access secrets.
- A compromised runner can steal code, tokens and deploy to production.
- Use isolated environments (Docker or Kubernetes).
- Restrict access using tags, protected runners and network policies.
- Regularly update and monitor runners.



Best Practices

- Use project or group runners for production workloads.
- Apply runner tags to control job assignment.
- Use Docker or Kubernetes for better isolation.
- Limit runner permissions and network access.
- Keep runners updated.
- Monitor runner usage and logs.



Key Takeaway

Runners are the workers that make GitLab CI/CD happen. Choose the right executor, secure your runners, use tags and control concurrency to build a fast, scalable and secure pipeline.



Common Mistakes

- Using shared runners for sensitive jobs.
- Not using tags.
- Running untrusted code on shell runners.
- No isolation (security risk).
- Too much concurrency (resource issues).
- Forgetting to update runners.
- Exposing secrets in job logs.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

Variables, Secrets, and Credentials



KEEP YOUR SECRETS SAFE. USE VARIABLES THE RIGHT WAY.

1. CI/CD Variables

- CI/CD variables store sensitive values and configuration for your pipelines.
- They can be used in jobs and scripts as environment variables.
- Defined at instance, group or project level.

Example usage in .gitlab-ci.yml:

```
deploy:
  stage: deploy
  script:
    - echo "Deploying to $ENVIRONMENT"
    - ./deploy.sh
  environment:
    name: $ENVIRONMENT
```

2. Project vs Group Variables

Feature	Project Variable	Group Variable
Scope	Available only in one project	Available in all projects in the group
Use case	Project-specific secrets (e.g. API keys)	Shared values (e.g. common registry credentials)
Management	Set in project settings	Set in group settings
Priority	Project variables take precedence	Used by all projects (if not overridden)



Tip:

Use group variables for common values across projects, and project variables for project-specific secrets.

3. Protected Variables

- Only available in protected branches and tags.
- Prevents exposure in feature branches or untrusted code.
- Commonly used for production credentials.



Example:

Set PROD_API_KEY as a protected variable so it is only available on main (protected branch).

4. Masked and Hidden Values

- Masked variables are not shown in job logs (the value is replaced with ****).
- Hidden variables are not visible in the GitLab UI after creation.
- Use masking for sensitive values like tokens, passwords or keys.

Example:

```
API_TOKEN = ****
(value is masked in logs)
```

5. File Variables

- File variables store large or multi-line secrets (e.g. SSH keys, certificates).
- GitLab creates a temporary file and provides the file path as a variable.

Example:

```
- cat $KUBE_CONFIG
- kubectl --kubeconfig
  $KUBE_CONFIG get pods
```

6. Environment Scope

- Variables can be limited to specific environments.
- Useful when you have different values for dev, staging and production.

Example:

Set a variable with environment scope:

- DEV_* → available only in dev
- STAGING_* → available only in staging
- PROD_* → available only in production

7. External Secret Managers

For better security, integrate GitLab with external secret managers instead of storing secrets in GitLab.

- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- CyberArk

Example (HashiCorp Vault):

```
secrets:
  VAULT_TOKEN:
    vault: production/ci/token
script:
  - export TOKEN=$VAULT_TOKEN
  - ./deploy.sh
```

8. Why Secrets Should Never Live in .gitlab-ci.yml

- ✗ The file is stored in your repository (even if private).
- ✗ It can be seen by anyone with repository access.
- ✗ It may be exposed in merge requests, forks or logs.
- ✗ It cannot be easily rotated or revoked.
- ✗ It increases the risk of accidental leaks.
- ✗ Use CI/CD variables or an external secret manager instead.



Never commit real secrets to Git.

Even if you delete them, they may still exist in history.



Good DevOps practices protect your secrets, your company and your future.



Secure Pipelines
Build Trust



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

Pipeline Rules and Conditional Execution



RUN THE RIGHT JOBS. SAVE TIME. SAVE COST. BUILD SMARTER.

1. What are Pipeline Rules?

- rules let you control when a job runs.
- They define conditions using **if**, **changes**, **exists** and more.
- Help you run only the necessary jobs.
- Reduce pipeline time and compute costs.
- More flexible and powerful than only/except (legacy).



Key Point:

Use rules to make your pipelines faster, cheaper and smarter.

2. Basic rules Syntax

```
job_name:
  script:
    - echo "Hello"
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on_success
    - when: never
```

Rules are evaluated from top to bottom. The first matching rule is used.

3. Common Conditions

Condition	Description
if	Evaluates a variable expression (e.g. branch, tag, pipeline source).
changes	Runs the job only if specific files have changed.
exists	Runs the job only if a file or directory exists.
when	Defines when to run the job (on_success, manual, never, etc).
allow_failure	Allows the job to fail without failing the pipeline.

4. Using if Conditions

Run a job only on the main branch.

```
deploy:
  script:
    - echo "Deploying..."
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on_success
    - when: never
```

5. Using changes

Run a job only when certain files change.

```
test:
  script:
    - echo "Running tests..."
  rules:
    - changes:
        - src/**
        - tests/**
      when: on_success
    - when: never
```

6. Using exists

Run a job only if a file exists in the repository.

```
security-scan:
  script:
    - echo "Running scan..."
  rules:
    - exists:
        - Dockerfile
      when: on_success
    - when: never
```

7. Pipeline Types



Branch pipelines

Run for commits on branches (e.g. main, dev).



Tag pipelines

Run when a Git tag is pushed.



Merge Request pipelines

Run for merge requests (source branch).



Scheduled pipelines

Run on a schedule (e.g. nightly builds, maintenance).

8. Examples for Different Pipeline Types

Branch pipeline (main only)

```
build:
  script:
    - echo "Building..."
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Tag pipeline

```
release:
  script:
    - echo "Creating release..."
  rules:
    - if: $CI_COMMIT_TAG
```

Merge Request pipeline

```
lint:
  script:
    - echo "Linting..."
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```

Scheduled pipeline

```
cleanup:
  script:
    - echo "Cleaning..."
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
```

9. Preventing Unnecessary Jobs

- Use **rules** to run jobs only when needed.
- Skip jobs for documentation-only changes.
- Use **changes** to detect relevant file modifications.
- Avoid running heavy jobs on every commit.
- Use **workflow: rules** to control entire pipelines.
- Combine conditions (branch, changes, exists) for precise control.
- Save compute resources and reduce costs.



Result: Faster pipelines, lower costs, and a more efficient CI/CD process.

10. Best Practices

- Use rules instead of only/except (deprecated).
- Be explicit with conditions.
- Use changes to skip unnecessary jobs.
- Combine multiple conditions when needed.
- Test your rules in a feature branch.
- Document your rules for your team.



Key Takeaway

Well-designed rules make your pipelines run only what's needed. This saves time, reduces costs and keeps your CI/CD efficient and scalable.



Common Mistakes

- Running all jobs on every commit.
- Not using changes or exists.
- Incorrect variable expressions.
- Forgetting merge request pipelines.
- Using outdated only/except syntax.
- Not testing rules before using in production.
- Over-complicated rules (hard to maintain).



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BETTER ENGINEERS. STRONGER SYSTEMS.

LEARN
PRACTICE
BUILD
GROW

Artifacts, Cache, and Pipeline Data Flow



MOVE DATA SMARTLY. BUILD FASTER. USE RESOURCES WISELY.

1. What are Artifacts?

- Artifacts are files generated by a job and saved by GitLab.
- They are stored in GitLab and can be downloaded or used by later jobs.
- Common artifacts: build outputs, test reports, packaged apps, Docker images (tar), etc.

Example:

```
build:
  stage: build
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
    expire_in: 1 week
```

2. What is Cache?

- Cache stores dependencies between pipeline runs to make jobs faster.
- Saved on the runner (not in GitLab).
- Used for things like npm, pip, Maven, Gradle, etc.
- Cache is not guaranteed and can be cleared anytime.

Example:

```
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - .npm/
    - node_modules/
  policy: pull-push
```

3. Key Differences

Feature	Artifacts	Cache
Purpose	Store job outputs (files)	Speed up jobs (dependencies)
Storage location	GitLab (persistent)	Runner (temporary)
Sharing	Available to later jobs in the same pipeline	Available to future pipelines (may be reused)
Use case	Build outputs, test reports, packaged files	Dependencies (npm, pip, Maven, etc.)
Retention	Configurable (expire_in)	No guaranteed retention
Reliability	Always available (within retention)	Best effort (can be cleared)

4. Dependencies Between Jobs

- By default, later stages can access artifacts from previous stages.
- Use dependencies to explicitly specify which jobs' artifacts to download.
- This reduces download time and storage usage.

Example:

```
test:
  stage: test
  dependencies:
    - build
  script:
    - npm test
```

5. Using needs (Faster Pipelines)

- needs allows a job to start as soon as its required jobs finish (instead of waiting for the entire previous stage).
- Speeds up pipelines significantly.

Example:

```
deploy:
  stage: deploy
  needs:
    - job: build
      artifacts: true
  script:
    - echo "Deploying..."
```

6. Artifact Retention

- Use `expire_in` to control how long artifacts are kept.
- After expiration, artifacts are automatically deleted.
- You can also keep important artifacts longer using GitLab settings.

Example:

```
artifacts:
  paths:
    - dist/
  expire_in: 2 weeks
```

7. Cache Keys

- Cache keys define when a cache should be used or refreshed.
- Use static keys, branch-specific keys or file-based keys.
- Yfile-based keys automatically update the cache when files change.

Examples:

```
# Static key
key: "node-cache"
# Branch-specific key
key: ${CI_COMMIT_REF_SLUG}
# File-based key
key:
  files:
    - package-lock.json
```

8. Dependency Caching

- Common for package managers:
 - npm → `node_modules/`
 - pip → `.cache/pip/`
 - Maven → `.m2/repository/`
 - Gradle → `.gradle/`
- Use `cache` to avoid downloading dependencies on every run.

Example (Python):

```
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - .cache/pip/
  policy: pull-push
```

9. Pipeline Data Flow (Example)



- Artifacts flow between jobs in the same pipeline.
- Cache is reused across pipeline runs.
- Use `needs` and `dependencies` to control the flow and reduce wait time.

10. Why Cache and Artifacts Solve Different Problems

Scenario	Use Artifacts	Use Cache
Pass build output to next job	✓	✗
Reuse dependencies in future pipelines	✗	✓
Store test reports	✓	✗
Speed up package installation	✗	✓
Keep files for download later	✓	✗

Key Takeaway

- Artifacts = share files between jobs (same pipeline).
- Cache = speed up future pipelines (dependencies).
- Use the right tool for the right problem.
- Combine both for fast, efficient and reliable pipelines.

Common Mistakes

- ✗ Using cache for build outputs.
- ✗ Not setting `expire_in` for artifacts.
- ✗ Using the same cache key for every branch.
- ✗ Forgetting to use dependencies or needs.
- ✗ Storing large files as cache (increases runner disk usage).

“ Smart pipelines don’t just run — they reuse, share, and scale. ”

— VERI QTA



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

GitLab Container Registry and Package Flow



BUILD ONCE. STORE SECURELY. DEPLOY ANYWHERE.

1. What is GitLab Container Registry?

- A built-in Docker container registry in GitLab.
- Stores your container images privately with your project or group.
- Integrated with GitLab CI/CD.
- Supports Docker, Helm charts and other packages.
- Keeps your images close to your code and your pipelines.



Your code.
Your images.
One platform.

2. Build Container Image

- Use a Dockerfile to build your image in CI/CD.
- The image contains your application and its dependencies.

Example (Dockerfile):

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

3. Tag Image

- Tags give your image a name and version.
- Use meaningful tags (version, branch, commit).
- Always tag with the commit SHA for immutability.

Example:

```
docker build -t $CI_REGISTRY_IMAGE:latest .
docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
docker build -t $CI_REGISTRY_IMAGE:v1.0.0 .
```

4. Authenticate to GitLab Registry

- Use the built-in CI/CD variables for authentication.
- GitLab provides a temporary token for each job.

Example:

```
echo "$CI_REGISTRY_PASSWORD" |
docker login -u "$CI_REGISTRY_USER"
--password-stdin $CI_REGISTRY
```

5. Push to GitLab Container Registry

- Push your tagged image to the registry in your project or group.
- The image is now stored and can be used in later stages (deploy, test, etc).

Example:

```
docker build -t $CI_REGISTRY_IMAGE:latest
docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
docker push $CI_REGISTRY_IMAGE:v1.0.0
```

6. Image Versioning

- Use clear versioning (semantic versioning or commit SHA).
- Common tags: latest, v1.0.0, \$CI_COMMIT_SHA.
- Helps with rollbacks and tracking changes.

Example tags:

```
latest      → always the latest build
v1.0.0     → release version
main       → from main branch
$CI_COMMIT_SHA → unique commit
```

7. Immutable Artifacts

- Container images should be immutable (do not change after build).
- Never overwrite the same tag (e.g. v1.0.0).
- Use new tags for new builds.



Immutable images =
predictable deployments,
safer rollbacks and better traceability.

8. Commit SHA Tagging

- Tag images with the commit SHA to ensure every build is uniquely identified.
- Helps with traceability and debugging.

Example:

```
docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```



Every commit produces a unique,
immutable image.

9. Artifact Promotion vs Rebuilding

- **Promotion:** Move the same image from dev → staging → production.
- **Rebuilding:** Building a new image for each environment.
- Prefer promotion to ensure the exact same artifact is tested and deployed.

Artifact Promotion (recommended)

- ✓ Same image across environments
- ✓ More reliable and consistent
- ✓ Faster and saves resources

Rebuilding (not recommended)

- ✗ Creates new images
- ✗ Risk of inconsistencies
- ✗ Slower and uses more compute

10. End-to-End Flow



Best Practices

- Use commit SHA tags.
- Keep images immutable.
- Use meaningful versioning.
- Use promotion instead of rebuilding.
- Clean up old images with retention policies.
- Restrict access to your registry.



Key Takeaway

The GitLab Container Registry lets you build, store and manage your container images securely. Use proper tagging, keep images immutable, and promote artifacts across environments for reliable and scalable deployments.



Common Mistakes

- ✗ Not tagging images.
- ✗ Overwriting the same tag.
- ✗ Storing secrets in Dockerfiles.
- ✗ Using latest in production.
- ✗ Rebuilding instead of promoting.
- ✗ Not cleaning up old images.
- ✗ Leaving the registry public.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQT

BETTER ENGINEERS. STRONGER SYSTEMS.

LEARN
PRACTICE
BUILD
GROW

Environments and Deployment Management



DEPLOY WITH CONFIDENCE. CONTROL YOUR ENVIRONMENTS. SHIP SAFELY.

1. What are GitLab Environments?

- GitLab environments represent a target where your application is deployed (e.g. development, staging, production).
- They help you track deployments, view history and manage releases.
- Each environment can have its own URL, variables, protection rules and deployment strategy.



**Environments =
Visibility + Control +
Safer Deployments.**

VERIQTA

2. Common Environment Types

Development

- Used for active development.
- Deployed frequently.
- May be auto-deployed from feature branches.

Staging

- Pre-production environment.
- Used for testing and validation.
- Should mirror production as closely as possible.

Production

- Live environment for end users.
- Requires strict controls and approvals.
- Changes should be carefully reviewed and monitored.

Use separate environments to reduce risk and catch issues early.

3. Defining Environments in .gitlab-ci.yml

```
deploy_staging:
  stage: deploy
  script:
    - echo "Deploying to staging..."
    - ./deploy.sh
  environment:
    name: staging
    url: https://staging.example.com

deploy_production:
  stage: deploy
  script:
    - echo "Deploying to production..."
    - ./deploy.sh
  environment:
    name: production
    url: https://example.com
    on_stop: stop_production
```

4. Environment Variables

- Each environment can have its own variables (different values per environment).
- Use environment-specific variables for things like API keys, database URLs and configuration.
- Set variables in GitLab UI under Settings → CI/CD → Variables and use environment scope.

Example:

```
API_URL=https://staging.example.com
DB_HOST=staging-db
API_URL=https://example.com
DB_HOST=prod-db
```

5. Deployment History

- GitLab keeps a history of all deployments for each environment.
- You can see who deployed, when, which commit and the status (success/failed).
- Helps with audit, rollback and troubleshooting.

Environments

	production https://example.com	View deployments
	staging https://staging.example.com	View deployments
	development https://dev.example.com	View deployments

6. Manual Deployment Gates

- Use manual jobs to require human approval before deploying (especially to production).
- Prevents accidental changes.
- You can restrict who can trigger manual jobs.

```
deploy_production:
  stage: deploy
  script:
    - ./deploy.sh
  environment:
    name: production
    when: manual
    allow_failure: false
```



Manual gates add an extra layer of safety for production deployments.

7. Environment Protection

- Protect environments (e.g. production) to limit who can deploy.
- Use GitLab's protected environments to allow only specific users or groups.
- Combine with approval rules for additional control.



Example:

Only members of the "Release Managers" group can deploy to production.

8. Deployment Ownership

- Assign clear ownership for each environment (team or individual).
- Define who is responsible for deployments, rollbacks and monitoring.
- Use CODEOWNERS, documentation and GitLab groups.



Clear ownership reduces confusion, improves accountability and leads to faster, safer deployments.

9. Best Practices

- ✓ Use separate environments (dev, staging, prod).
- ✓ Keep environment configuration in GitLab.
- ✓ Use environment-specific variables.
- ✓ Protect production environments.
- ✓ Use manual gates for critical deployments.
- ✓ Monitor deployment history.
- ✓ Document ownership and rollback procedures.
- ✓ Keep staging as close to production as possible.



Common Mistakes

- ✗ Deploying directly to production.
- ✗ No environment protection.
- ✗ Using the same variables for all environments.
- ✗ Skipping manual approvals.
- ✗ No clear ownership or documentation.
- ✗ Not tracking deployment history.



**Right environment. Right controls.
A safer path to production.**

— VERIQTA



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD TODAY. DEPLOY TOMORROW. GROW ALWAYS.

**LEARN
PRACTICE
BUILD
GROW**

Production Deployment Strategies



DEPLOY SMART. REDUCE RISK. KEEP YOUR USERS HAPPY.

1. Rolling Deployments

- Update instances gradually without downtime.
- Old and new versions run together during the rollout.
- Good for most applications.
- Reduces risk but not as safe as blue-green or canary.

Rolling Deployment



VERIQTA

2. Blue-Green Deployment

- Two identical environments (blue and green).
- Deploy new version to the idle environment.
- Switch traffic from blue → green instantly.
- Easy and fast rollback.

Blue-Green Deployment



VERIQTA

3. Canary Deployment

- Release to a small percentage of users first.
- Monitor metrics and errors.
- Gradually increase traffic if healthy.
- Minimizes risk and user impact.

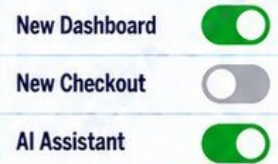
Canary Deployment



VERIQTA

4. Feature Flags

- Deploy code but keep features disabled.
- Enable features for a small group of users.
- Test in production without full release.
- Quickly turn off if issues arise.



VERIQTA

5. Manual Approvals

- Require human approval before deploying to sensitive environments (e.g. production).
- Adds an extra layer of control and accountability.
- Common in regulated industries.

Example:

```

deploy_prod:
  stage: deploy
  when: manual
  allow_failure: false
  
```

VERIQTA

6. Deployment Freeze

- Temporary pause on deployments (e.g. during holidays, major events).
- Prevents risky changes during critical periods.
- Can be enforced using pipeline rules or GitLab settings.



Example Period:
Dec 20 – Jan 5
(No deployments)

VERIQTA

7. Rollback

- Revert to a previous working version if issues occur.
- Use GitLab's deployment history or redeploy a previous image/tag.
- Should be tested regularly to ensure it works.



Example:
Redeploy image:
v1.2.3

VERIQTA

8. Release Safety

- Use automated tests, monitoring and gradual rollouts.
- Validate health metrics before full rollout.
- Have a rollback plan ready.
- Use feature flags and canary deployments.



Safe releases protect your users, your business and your reputation.

VERIQTA

9. Separating Deployment from Release

- Deployment** = make the code available in an environment (e.g. deploy to production).
- Release** = make the feature available to users (e.g. enable via feature flag).
- This allows you to deploy early, test in production, and release when ready.



VERIQTA

10. Strategy Comparison

Strategy	Downtime	Risk Level	Rollback Ease	Best Use Case
Rolling	No	Medium	Medium	General use
Blue-Green	No	Low	Easy	Critical systems
Canary	No	Low	Easy	New features
Feature Flags	No	Low	Easy	Gradual feature release
Manual Approvals	N/A	Low	N/A	Sensitive environments
Deployment Freeze	N/A	Low	N/A	High-risk periods

VERIQTA



A good deployment strategy is not about speed, but about control, safety and trust.

— VERIQTA



Key Takeaway

Deploy often, release safely, and always be ready to roll back.

VERIQTA



Common Mistakes

Deploying without testing.
No rollback plan.
Skipping manual approvals.
Releasing during freeze periods.
Not using feature flags.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN. BUILD. DEPLOY. GROW.

LEARN
PRACTICE
BUILD
GROW

GitLab Security and DevSecOps



SECURE EARLY. BUILD CONFIDENCE. SHIP FASTER.

1. What is DevSecOps?

- Integrates security into every stage of the development lifecycle.
- Finds and fixes issues early.
- Reduces risk, improves quality and builds trust.
- GitLab provides built-in security tools that run directly in your pipelines.



Security is everyone's responsibility, not just the security team.

VERIQTA

2. SAST (Static Application Security Testing)

- Analyzes source code for security vulnerabilities.
- Finds issues like injection, unsafe functions and misconfigurations.
- Runs automatically in the CI pipeline.

```
sast:
  stage: test
  script:
    - echo "Running SAST..."
  include:
    - template: Security/SAST.gitlab-ci.yml
```

VERIQTA

3. DAST (Dynamic Application Security Testing)

- Tests the running application for security issues.
- Finds vulnerabilities that only appear at runtime.
- Works best in staging environments.

```
dast:
  stage: test
  script:
    - echo "Running DAST..."
  include:
    - template: Security/DAST.gitlab-ci.yml
```

VERIQTA

4. Dependency Scanning

- Checks third-party libraries and packages for known vulnerabilities.
- Supports many languages (e.g. npm, pip, Maven, Gradle).
- Helps you detect and upgrade vulnerable dependencies early.

```
dependency_scanning:
  stage: test
  include:
    - template: Security/Dependency-Scanning.gitlab-ci.yml
```

VERIQTA

5. Secret Detection

- Scans your code for leaked secrets (API keys, tokens, passwords, etc.).
- Prevents accidental exposure of sensitive data.
- Runs in the pipeline and can block merges if secrets are found.

```
secret_detection:
  stage: test
  include:
    - template: Security/Secret-Detection.gitlab-ci.yml
```

VERIQTA

6. Container Scanning

- Scans container images for vulnerabilities.
- Checks OS packages and application dependencies inside the image.
- Helps you ship safer containers to production.

```
container_scanning:
  stage: test
  include:
    - template: Security/Container-Scanning.gitlab-ci.yml
```

VERIQTA

7. Software Supply-Chain Risk

- Vulnerabilities can come from open-source components, docker images and CI/CD dependencies.
- Use dependency scanning, container scanning and license compliance.
- Verify the source and integrity of third-party components.



Know what you use.
Trust what you build.

VERIQTA

8. Security Findings in Pipelines

- Security scan results appear directly in the pipeline.
- You can view details in the Security Dashboard.
- Findings include severity (Critical, High, Medium, Low).
- Can be configured to fail the pipeline based on policy.

Security scan results

Critical	2
High	5
Medium	8
Low	12

VERIQTA

9. Security Gates Without Destroying Velocity

- Use risk-based policies (block only critical/high issues).
- Allow lower severity issues to pass with warnings.
- Use manual approvals for production deployments.
- Combine automated scans with developer education.
- Continuously improve and reduce false positives.



The goal is secure and fast delivery — not security or speed, but both.

VERIQTA

10. Example: Full Security Pipeline

```
stages:
  - test
  - build
  - deploy
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/DAST.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml
  - template: Security/Container-Scanning.gitlab-ci.yml
```

VERIQTA

Key Takeaways

- Security early reduces risk.
- Automate security in pipelines.
- Use the right tools for the right risks.
- Block critical issues, not productivity.
- Build a secure, trusted and scalable delivery process.

VERIQTA



Common Mistakes

- Ignoring security scan results.
- Storing secrets in the repository.
- Not updating vulnerable dependencies.
- Disabling security jobs to save time.
- No clear policy for blocking/allowing builds.
- Assuming open-source packages are always safe.



Best Practices

- Enable all relevant security scans.
- Keep dependencies and base images updated.
- Use environment-specific security policies.
- Review and fix findings regularly.
- Educate your team on secure coding practices.
- Monitor the Security Dashboard.



A secure pipeline builds more than software. It builds trust.

— VERIQTA



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BETTER ENGINEERS. SAFER SYSTEMS. BRIGHTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW

Permissions, RBAC, and Protected Resources



THE RIGHT ACCESS. THE RIGHT CONTROL. A SAFER GITLAB.

1. GitLab Roles (RBAC)

- GitLab uses Role-Based Access Control (RBAC) to manage what users can do.
- Roles are assigned at project, group or instance level.
- Choose the lowest role that allows the user to do their work (least privilege).
- Roles can be combined with groups for easier management.



Right people. Right access.
Stronger projects.

VERIQTA

2. GitLab Access Roles and Permissions

Role	Key Permissions	Use Case
 Guest	- Can view the project - No write access	External users who only need to view
 Planner	- Can create issues - Can create merge requests - Cannot push code	Product managers, team planners
 Reporter	- Can view code, issues, pipelines - Cannot make changes	Auditors, QA, stakeholders
 Developer	- Can push code - Can create merge requests - Can run pipelines	Engineers working on the project
 Maintainer	- Full project management (except owner rights) - Can manage settings, CI/CD, members	Tech leads, senior engineers
 Owner	- Full control - Can manage billing, rename or delete project	Project creator or administrators

VERIQTA

3. Protected Branches

- Restrict who can push and merge to important branches (e.g. main).
- Prevent direct pushes.
- Require merge requests and approvals.
- Can require passing pipelines before merge.

Example:

- Protect main branch
- Allow merge: Maintainers only
 - Require approvals: 2
 - Require pipeline to pass: Yes

VERIQTA

4. Protected Tags

- Restrict who can create or update tags (e.g. release tags).
- Prevent unauthorized releases from being published.

Example:

- Protect v* tags
- Allowed to create: Maintainers only

VERIQTA

5. Protected Environments

- Control who can deploy to sensitive environments (e.g. production).
- Require approvals before deployment.
- Limit which branches or users can deploy.

Example:

- Protect production environment
- Allowed to deploy: Maintainers only
 - Require approval: Yes

VERIQTA

6. Least Privilege Principle

- Give users only the access they need to do their job.
- Reduces the risk of accidental or malicious changes.
- Review access regularly and remove unused permissions.
- Use groups and roles instead of giving individual high-level access.



Less access =
less risk =
more security.

VERIQTA

7. Preventing Unauthorized Production Changes

- Use protected branches (e.g. main).
- Require merge requests and code reviews.
- Enforce passing pipeline checks.
- Use protected environments for production.
- Limit deployment permissions to trusted users only.
- Enable audit logs to track changes.

VERIQTA

8. Real-World Example



VERIQTA



Control access today,
protect your tomorrow. — VERIQTA



Common Mistakes

- ✗ Giving everyone Developer or Maintainer access.
- ✗ Not protecting the main branch.
- ✗ Allowing direct pushes to production.
- ✗ Not using protected environments.
- ✗ No approval process for deployments.
- ✗ Not reviewing and removing unused access.

VERIQTA



Key Takeaways

- ✓ Use the right role for the right person.
- ✓ Protect branches, tags and environments.
- ✓ Follow least privilege.
- ✓ Require approvals for production changes.
- ✓ Review access regularly.
- ✓ Security and speed can work together.

VERIQTA



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

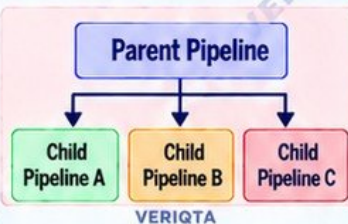
Advanced Pipeline Architecture

SCALE SMART. BUILD FLEXIBLE PIPELINES. DELIVER FASTER.



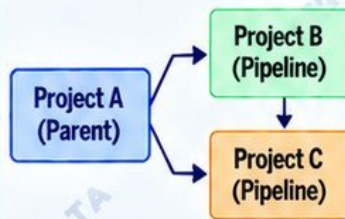
1. Parent-Child Pipelines

- A parent pipeline triggers one or more child pipelines.
- Helps break large pipelines into smaller, manageable units.
- Each child pipeline has its own .gitlab-ci.yml.
- Useful for monorepos and complex projects.



2. Multi-Project Pipelines

- Trigger pipelines in other projects (e.g. shared templates, infra repos).
- Useful for microservices and separate repositories.
- Allows independent teams to work while keeping dependencies.



3. Includes

- Reuse CI configuration from other files or projects.
- Include local, remote or template files.
- Helps keep pipelines clean and DRY (Don't Repeat Yourself).

Example:

```
include:
  - local: '/templates/build.yml'
  - project: 'group/ci-templates'
    file: '/deploy.yml'
  - remote:
      url: 'https://example.com/ci.yml'
```

4. Reusable CI Components

- Use CI/CD components to share reusable jobs, stages and templates.
- Store in a central repository and include in multiple projects.
- Improves consistency and reduces maintenance.



Write once.
Use everywhere.
Scale with
CI components.

5. Dynamic Pipelines

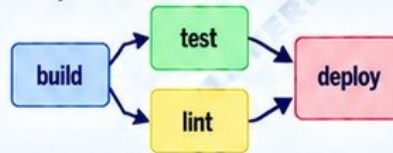
- Generate pipelines dynamically based on changes, variables or external data.
- Useful for monorepos and config-driven setups.
- Allows creating jobs and stages at runtime.

Example:

```
generate-pipeline:
  stage: build
  script: python generate.py
  artifacts:
    paths: [generated-ci.yml]
```

6. DAG Pipelines (Directed Acyclic Graph)

- Run jobs in parallel based on dependencies instead of stages.
- Faster execution and better resource utilization.
- Jobs start as soon as their dependencies are met.



Jobs run in parallel where possible.

7. needs

- Defines explicit job dependencies.
- Allows jobs to start earlier without waiting for the entire stage.
- Speeds up pipeline execution.

Example:

```
test:
  stage: test
  needs: ["build"]
  script: echo "Running tests"
```

8. Pipeline Modularity

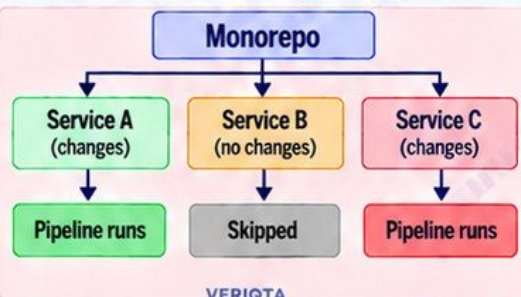
- Split configuration into smaller, reusable pieces.
- Use includes, components, and child pipelines.
- Makes pipelines easier to maintain, test and scale.
- Supports team ownership of different parts.



Modular pipelines
are cleaner, faster
to maintain and
easier to scale.

9. Monorepo Considerations

- Use dynamic pipelines to run only affected services.
- Detect changes using rules: changes.
- Split pipelines by directory or service.
- Use child pipelines for each component.
- Optimize with caching and parallel jobs.



Key Takeaways

- ✓ Use parent-child pipelines for large and complex projects.
- ✓ Leverage multi-project pipelines for cross-repository workflows.
- ✓ Keep your CI configuration modular and reusable.
- ✓ Use DAG and needs to speed up pipelines.
- ✓ Generate dynamic pipelines for flexibility.
- ✓ Optimize monorepos to run only what changed.



A well-designed pipeline architecture gives you flexibility, speed and scale without the chaos.

— VERIQTA



Common Mistakes

- ✗ Creating one giant .gitlab-ci.yml file.
- ✗ Not using includes or reusable components.
- ✗ Running all jobs even when nothing changed.
- ✗ Ignoring pipeline dependencies.
- ✗ Overusing stages instead of DAG.
- ✗ Not optimizing monorepo pipelines.
- ✗ Making pipelines hard to maintain.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD SKILLS. SHARE KNOWLEDGE. GROW TOGETHER.

LEARN
PRACTICE
BUILD
GROW

GitLab CI/CD Troubleshooting



FIND ISSUES FASTER. FIX WITH CONFIDENCE. KEEP DELIVERY ON TRACK.

1. Pipeline Does Not Start

- Check .gitlab-ci.yml syntax.
- Verify pipeline rules and conditions.
- Ensure the commit triggers a pipeline (branch, tag, MR).
- Check if CI/CD is enabled for the project.
- Look for [ci skip] in the commit message.

Check pipeline configuration:

CI Lint → GitLab UI
or
`curl --header "PRIVATE-TOKEN: <token>" \`
`"https://gitlab.com/api/v4/ci/lint"`

VERIQTA

2. Job Remains Pending

- No available runners.
- Runner is offline.
- Runner tags do not match.
- Runner is busy with other jobs.
- Check job requirements (tags, protected, specific runner).

Check runners:

Settings → CI/CD → Runners
OR
`gitlab-runner list`

VERIQTA

3. Runner Unavailable

- Runner is not registered.
- Runner is offline or unhealthy.
- Incorrect runner URL or token.
- Runner version is incompatible.
- Runner not allowed for protected branches.

Verify runner status:

`gitlab-runner status`
`gitlab-runner verify`
`gitlab-runner run`

VERIQTA

4. Job Fails Unexpectedly

- Check job logs for error details.
- Verify script commands.
- Check environment variables.
- Ensure required tools and dependencies are available.
- Look for network, permission or timeout issues.



View logs:

GitLab UI → Job → Trace
OR
`gitlab-runner --debug`

VERIQTA

5. Artifact Missing

- Check if artifacts are defined in .gitlab-ci.yml.
- Verify paths are correct.
- Ensure the job completed successfully.
- Check artifact retention period.
- Confirm downstream jobs are using dependencies or needs.

Example:

artifacts:
paths:
- build/
expire_in: 1 week



VERIQTA

6. Cache Misses

- Check cache key configuration.
- Ensure cache paths are correct.
- Verify cache policy (pull, push, pull-push).
- Cache is per branch by default.
- Large or frequently changing dependencies can cause misses.

Example:

cache:
key: \${CI_COMMIT_REF_SLUG}
paths:
- .m2/repository/
policy: pull-push



VERIQTA

7. Permission Failures

- User does not have required role.
- Protected branch or environment restrictions.
- Missing access to variables, secrets or Docker registry.
- Runner lacks required permissions.
- Check group/project permissions and access tokens.



Check your role:

Settings → Members
and review protected settings.

VERIQTA

8. Docker Build Failures

- Check Dockerfile syntax.
- Ensure base images are accessible.
- Verify network access (proxies, firewalls).
- Check disk space on runner.
- Use correct Docker version and build arguments.

Example (use detailed logs):

`docker build --progress=plain \`
`-t myapp:${CI_COMMIT_SHORT_SHA} .`



VERIQTA

9. Deployment Failures

- Check deployment script and logs.
- Verify environment variables and secrets.
- Ensure correct credentials and permissions.
- Check infrastructure (cluster, server, cloud).
- Look for rollout, health check or DNS issues.

Common checks:

- `kubectl get pods`
- `kubectl describe deployment`
- Check application logs
- Verify service and ingress



VERIQTA

10. Debug Flow: Pipeline → Job → Runner → Executor → Workload



VERIQTA

VERIQTA

VERIQTA

VERIQTA

VERIQTA



Troubleshooting is a skill.
Logs tell the story.
Follow the evidence.

— VERIQTA



Key Takeaways

- Start from the pipeline and go deeper.
- Use logs at every level.
- Check configuration, permissions and infrastructure.
- Fix the root cause, not just the symptom.
- Document common issues and solutions.



Common Mistakes

- Not reading the full job logs.
- Ignoring runner health.
- Misconfigured tags or permissions.
- Wrong cache or artifact paths.
- Assuming it's a GitLab issue (often it's config).
- Not checking environment variables.
- Skipping network and infrastructure checks.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN
PRACTICE
BUILD
GROW

DEBUG TODAY. DEPLOY TOMORROW. GROW ALWAYS.

GitLab Reliability and Scaling

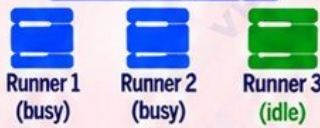


BUILD RELIABLE PIPELINES. SCALE WITH CONFIDENCE.

1. Runner Capacity

- Ensure you have enough runners for your workloads.
- Match runner types (shared, specific, GPU, etc.) to your jobs.
- Monitor runner utilization and idle capacity.
- Use autoscaling for variable workloads.

Runner Capacity



VERIQTA

2. Queue Time

- Queue time is how long a job waits for a runner.
- High queue time indicates insufficient runners or wrong tags.
- Use more runners or adjust tags.
- Monitor queue time trends to prevent delays.

Example:

Long queue time
Job waits 8 minutes
before starting.



VERIQTA

3. Pipeline Duration

- Measure total pipeline duration from start to finish.
- Long pipelines slow down delivery.
- Optimize by parallelizing jobs, using needs, and caching dependencies.
- Track duration over time to spot regressions.

Example:

Before optimization: 25 min
After optimization: 8 min



VERIQTA

4. Concurrency

- Control how many pipelines or jobs run at the same time.
- Avoid overloading runners and infrastructure.
- Use resource groups to limit concurrent deployments.
- Balance speed with stability.

Concurrency Control



VERIQTA

5. Resource Saturation

- Runners can run out of CPU, memory, disk or network.
- Resource saturation causes slow jobs or failures.
- Monitor runner resource usage in real time.
- Use autoscaling or larger instances when needed.

Watch for:



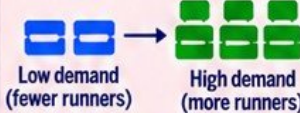
- High CPU usage
- High memory usage
- Disk space running low
- Network saturation

VERIQTA

6. Runner Autoscaling

- Automatically scales runners based on demand.
- Useful for cloud and Kubernetes runners.
- Reduces queue time and handles spikes.
- Scales down when not needed to save costs.

Autoscaling Runners

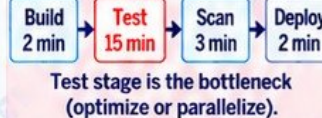


VERIQTA

7. Pipeline Bottlenecks

- Identify slow stages or jobs.
- Common bottlenecks: long tests, builds, security scans, or external dependencies.
- Use DAG and needs to run jobs in parallel.
- Optimize scripts, caching, and artifacts.

Find the bottleneck



VERIQTA

8. Artifact / Storage Growth

- Artifacts consume storage over time.
- Large artifacts can increase costs and slow down pipelines.
- Set artifact expiration and cleanup policies.
- Monitor storage usage regularly.

Example:

Keep artifacts only for 7 days or latest successful pipeline.



VERIQTA

9. Dependency Failures

- Pipelines can fail due to external dependencies (e.g. package registries, APIs, cloud services).
- Network issues, rate limits, or service outages can cause job failures.
- Use retries, timeouts, and fallback strategies.
- Monitor the health of critical dependencies.

Common dependencies:

- Package registries (npm, PyPI, Maven)
- Container registries
- Cloud providers (AWS, Azure, GCP)
- External APIs and services



VERIQTA

10. Metrics That Reveal CI/CD Degradation

Metric	What It Tells You	Warning Sign
Queue time	Runner availability	Increasing trend
Pipeline duration	Overall performance	Pipelines getting slower
Runner utilization	Resource usage	Runners always busy
Job failure rate	Stability	More failed jobs
Artifact storage size	Storage growth	Rapid increase
Cache hit rate	Cache efficiency	Low cache hit rate
Concurrency limits	Scaling constraints	Jobs waiting or blocked
External dependency uptime	Service health	Frequent timeouts/failures

VERIQTA



A reliable pipeline is a fast pipeline.
A scalable pipeline is a future-ready pipeline.

— VERIQTA



Key Takeaways

- Monitor your CI/CD metrics.
- Scale runners based on demand.
- Identify and remove bottlenecks.
- Keep storage and dependencies healthy.
- Reliability and speed go together.



Common Mistakes

- Not monitoring queue time.
- Undersized runner capacity.
- Ignoring storage growth.
- No autoscaling strategy.
- Not tracking dependency failures.
- Overlooking pipeline metrics.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LEARN. PRACTICE. BUILD. GROW.

LEARN
PRACTICE
BUILD
GROW

Production GitLab Project, Commit to Deployment



FROM CODE TO CUSTOMERS. A COMPLETE REAL-WORLD FLOW.

1. Developer Pushes Code



- Developer commits code to a feature branch.
- Code is pushed to GitLab repository.
- Triggers pipeline (if configured).

```
git push origin feature-branch
```

2. Merge Request Created



- Developer creates a Merge Request (MR).
- Describes changes, links issues.
- MR triggers pipeline and review process.

VERIQTA

3. Automated Tests Run



- CI pipeline runs unit tests, integration tests, linting.
- Fails fast if tests fail.
- Provides feedback to the MR.

VERIQTA

4. Security Scans Execute



- Runs SAST, dependency scanning, secret detection, container scanning (if applicable).
- Finds vulnerabilities early.
- Blocks merge if critical issues are found (based on policy).

VERIQTA

5. Review and Approval



- Team reviews code, test results, and security findings.
- Approvals required (especially for protected branches).
- Discuss, request changes if needed.

VERIQTA

6. Merge to Protected Branch



- After approval, MR is merged to a protected branch (e.g. main).
- Branch protection enforces rules (required approvals, passing pipelines, etc.).

VERIQTA

7. Build Immutable Container Image



- CI pipeline builds container image (e.g. Docker).
- Image is tagged with commit SHA and version.
- Image is immutable (cannot be changed).

```
docker build -t app:$CI_COMMIT_SHA
```

VERIQTA

8. Push Image to Registry



- Push the built image to GitLab Container Registry.
- Use secure authentication (CI_JOB_TOKEN).
- Image is stored with version tags (commit SHA, semver).

```
docker push $CI_REGISTRY_IMAGE
```

VERIQTA

9. Deploy to Staging



- Deploy the image to staging environment (using CI/CD).
- Use environment variables and configuration.
- Run migration scripts (if needed).

VERIQTA

10. Validate



- Run smoke tests and end-to-end tests.
- Verify application functionality.
- Check logs, metrics and health.
- Confirm everything works as expected in staging.

VERIQTA

11. Production Approval



- Request production deployment approval (manual gate).
- May require additional approvals (e.g. release manager, change board).

VERIQTA

12. Deploy Production



- Deploy the same immutable image to production.
- Use automated or manual deployment (based on policy).
- Follow deployment strategy (rolling, blue-green, canary, etc.).

VERIQTA

13. Observe Deployment



- Monitor application health, logs, metrics, and alerts.
- Verify SLIs/SLOs are healthy.
- Check for errors, latency or abnormal behavior.

VERIQTA

14. Simulate Failure



- Test what happens when something goes wrong (e.g. bad deployment, service failure).
- Validate alerting and detection.
- Ensure rollback process works.

VERIQTA

15. Roll Back



- Roll back to previous stable version (using GitLab or deployment tool).
- Verify service is restored.
- Communicate with team and stakeholders.

VERIQTA

16. Verify Recovery



- Confirm application is healthy after rollback.
- Run tests and check metrics.
- Document the incident and lessons learned.
- Improve process if needed.

VERIQTA

Complete Production Flow



A reliable deployment is not the absence
A failure, but the ability to recover quickly.

— VERIQTA

VERIQTA



Key Takeaways

- Automate as much as possible.
- Use immutable artifacts.
- Enforce security and quality gates.
- Deploy the same image from dev to production.
- Monitor, be ready to rollback.
- Continuous improvement makes you stronger.



Common Mistakes

- Skipping tests or security scans.
- Merging without proper review.
- Using mutable images.
- Not using protected branches.
- No monitoring after deployment.
- No rollback plan.
- Ignoring lessons learned.

VERIQTA



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

BUILD KNOWLEDGE. DEPLOY CONFIDENCE. GROW TOGETHER.

LEARN
PRACTICE
BUILD
GROW